École Supérieure Privée d'Ingénierie
et de Technologies (ESPRIT)

Al Baraka Bank Tunisia
Centre Urbain Nord, 1082 Tunis

EXECUTIVE SUMMARY & CLOSING HANDOVER REPORT

A Governed Intelligence for Islamic Banking

The Integrated RAGLab and Bank Assistant Platform

HANDOVER PURPOSE

Condensed technical synthesis delivered to Al Baraka Bank Tunisia at the close of the engineering internship: architecture, empirical evaluation, governance guarantees, security posture, and pilot roadmap.

PREPARED BY
Farah AYARI
Software Engineering, ESPRIT

HOST ORGANISATION
Al Baraka Bank Tunisia
Av. Cheikh Mohamed Fadhel Ben Achour,
Centre Urbain Nord, 1082 Tunis

COMPANY SUPERVISOR
Mrs. Ahlem Ben Haddoud
Al Baraka Bank Tunisia

INTERNSHIP PERIOD
03 August – 03 October 2026
Academic Year 2025–2026

OVERVIEW

Executive Brief

A generative model answers even when it does not know. A regulated bank requires a system that proves every statement from approved texts—and refuses cleanly when evidence is missing.

In Islamic banking, product rules, profit-sharing conditions, deadlines, ceilings and sanctions are governed simultaneously by national banking law, Central Bank circulars and Sharia compliance procedures. Deploying an unguarded large language model over such material creates an unacceptable legal and reputational exposure: fluent prose hides fabricated rates or misattributed clauses.

This handover report summarises the platform designed, built, measured and packaged during the internship at Al Baraka Bank Tunisia. The deliverable comprises two strictly separated cooperating systems:

- **RAGLab (Answering Engine)** — a four-layer retrieval-augmented generation microservice built over four repaired, sponsor-approved Arabic banking documents (339 structural chunks and a 198-unit legal knowledge base). Every claim rests on verifiable source evidence, every numeric value is deterministically checked against the cited passage, and insufficient evidence produces a structured refusal rather than speculation.
- **Bank Assistant (Web & Edge Platform)** — a trilingual customer portal (Arabic, French, English with RTL support) and a French staff supervision console backed by Spring Boot 4, Java 21 virtual threads and PostgreSQL 16. The edge service is the sole writer to the bank database, enforces quotas and 9-permission RBAC, masks customer context per request, and strips all internal provenance from customer-facing answers while preserving full auditability for staff.

+20 pts	100 %	1.000	6 / 6
live hit@1 gain from structural chunking (51 % → 71 %)	law questions ranked 1 via the 198-unit knowledge layer	cited-source overlap across two independent answer models	adversarial & injection scenarios blocked by the citation gate

LIMITATION **Operational readiness scope**

The platform is engineered and validated for a **supervised internal or controlled pilot** on approved, non-confidential banking documents. Moving to unattended customer-facing production requires closing the pre-pilot compliance items detailed in Section 6 (input-side PII scrubbing, data-retention jobs, and formal sub-processor or on-premise model hosting decisions).

NOTE **Turnkey handover state**

All deliverables—source code, Flyway SQL migrations, frozen 60-question trilingual benchmarks, automated CI pipelines, published Docker Hub images (`elayarifarah/albaraka-*`), and the zero-build `albaraka-demo.zip` bundle—are handed over ready for immediate internal deployment.

OVERVIEW

Contents & Handover Index

1 Regulatory Context, Corpus & Scope 4
- 1.1 Islamic Banking Framing at Al Baraka Bank Tunisia . . . 4
- 1.2 Objectives, Deliverables & Scope Boundaries . . . 4

2 Platform Architecture & Governance 6
- 2.1 End-to-End Topology & Division of Ownership . . . 6
- 2.2 Lifecycle of a Customer Question & Core Technology Stack . . . 6

3 RAGLab: The Four-Layer Answering Engine 8
- 3.1 Layer 1 — Linguistic Corpus Repair, Structural Chunking & Self-Describing Index . . . 8
- 3.2 Layer 2 — Declarative Knowledge Layer on Law n° 2016-48 . . . 8
- 3.3 Layers 3 & 4 — Sufficiency, Citation Gate, Refusal Taxonomy & Service Contract . . . 9

4 Bank Assistant: Edge, Portal & Console 10
- 4.1 Identity Separation, Two-Phase Quota Metering & Fault-Tolerant Logging . . . 10
- 4.2 Ephemeral Customer Context Masking & Staff Back-Office Console . . . 10

5 Empirical Evaluation & Quality Gates 12
- 5.1 Benchmark Protocol & Structure-Aware Chunking Gain (+20 pts hit@1) . . . 12
- 5.2 Dense vs. Hybrid Retrieval, Legal Knowledge Index & Answer-Model Independence . . . 12

6 Security, Operations & Pilot Roadmap 14
- 6.1 Trust Boundaries, OWASP LLM Defences & Tunisian Personal-Data Assessment . . . 14
- 6.2 Containerised Handover Package & Operational Lessons . . . 15
- 6.3 Recommended Closing Roadmap for Al Baraka Bank Tunisia . . . 15

HANDOVER PACKAGE AT A GLANCE

Deliverable	Artifact / Location	Purpose for Al Baraka Bank Teams
Answering Engine	`raglab/` (port `8000`)	4-layer FastAPI RAG microservice, repaired corpus & 198-unit Law index.
Edge & Web Portals	`bank-assistant/` (port `8080`)	Spring Boot 4 + Angular 21 customer portal (AR/FR/EN) & staff console.
Standalone Demo	`albaraka-demo.zip`	Image-only 4-container Docker Compose bundle (`docker compose up -d`).
Docker Registry	`elayarifarah/albaraka-*`	Pre-built `raglab`, `backend`, and `frontend` images on Docker Hub.
Quality & Audit	CI workflows & `/eval`	819+ automated tests/checks, 20-point smoke test, JSONL & DB audit trails.

SECTION 1

1

Regulatory Context, Corpus & Scope

How Tunisia's Islamic banking framework shapes the technical requirements of a banking assistant, which four documents form the authoritative corpus, and what the project commits to deliver.

1.1 Islamic Banking Framing at Al Baraka Bank Tunisia

Established in 1983 and operating as a resident universal bank across 44 Tunisian branches, Al Baraka Bank Tunisia conducts interest-free financing, investment deposits and banking services under two foundational regulatory texts, both included directly in the platform's corpus:

- **Law n° 2016-48 of 11 July 2016** on banks and financial institutions: Article 11 defines Islamic banking operations—specifically **Mourabaha** (المرابحة), **Ijara** (الإجارة), **Istisna'a** (الاستصناع), **Salam** (السلم), **Moudaraba** (المضاربة), **Moucharaka** (المشاركة) and **investment deposits** (ودائع الاستثمار)—while Article 54 establishes Sharia compliance oversight.
- **BCT Circular n° 2019-08 of 14 October 2019**: defines the operational conditions, contractual prerequisites and governance rules of each Islamic financing and deposit instrument.

Three practical difficulties arise when answering customer or staff questions from these texts:

1. **Lexical gap between everyday and legal wording:** a customer asks in plain words how to finance shop equipment, whereas the regulatory text specifies contractual forms (الصيغة) and asset ownership rules (محل العقد).
2. **Asymmetric multilingualism:** the authoritative corpus is written entirely in Arabic, yet inquiries arrive in Arabic, French and English; cross-lingual retrieval must bridge French/English queries directly to Arabic legal passages.
3. **High legal cost of numerical imprecision:** ratios, notice periods, ceilings and financial penalties are legally binding; paraphrasing from model memory is unacceptable.

Table 1.1 The four real Arabic documents composing the governed corpus.

Document	Nature	Authoritative Role in the Platform	Chunks
Law n° 2016-48 (11 July 2016)	National law	Highest legal authority: 198 articles across 11 titles	233
BCT Circular n° 2019-08	BCT circular	Operational conditions of Islamic banking contracts (20 sections)	13
Internal Islamic Operations Guide	Internal policy	Bank compliance procedures and operational workflows	54
Islamic Finance Primer (*Madkhal*)	Training manual	Conceptual definitions, principles and historical benchmarks	39
Total Approved Corpus		**294 audited linguistic repairs; zero synthetic documents**	**339**

1.2 Objectives, Deliverables & Scope Boundaries

Every architectural decision followed a strict **step-and-plan protocol**: written specification, offline verification, CI validation, and paired measurement on frozen question sets before changing any default. Table 1.2 maps the seven initial internship objectives to the delivered handover state.

Table 1.2 Internship objectives and closing delivery status.

ID	Objective	Delivered Handover Outcome
O1	End-to-end Arabic RAG pipeline	Delivered: audited linguistic repair, structure-aware chunking, Nemotron 2048-d embeddings, ChromaDB index, retrieval, grounded synthesis and verification.
O2	Reproducible models & metrics	Delivered: pinned embedding and answer models with no silent fallback; every active default selected via paired benchmark rules.
O3	First-class honest refusal	Delivered: six deterministic refusal codes on the engine; bank-approved trilingual guidance substituted for customers at the edge.
O4	Integrated service contract	Delivered: frozen, versioned 27-route FastAPI microservice (`X-Service-Token`) consumed as the sole engine of Bank Assistant.
O5	Instrumented quality & audit	Delivered: 472+ unit tests & 133 offline checks (RAGLab), 214 web platform tests, frozen 50+10 question benchmarks, JSONL & DB audit logs.
O6	Customer portal & staff console	Delivered: trilingual RTL-ready customer portal + French back-office for live monitoring, RBAC administration and runtime RAGLab control.
O7	Zero customer PII in index	Delivered by construction: customer records stay in PostgreSQL; only an ephemeral masked snapshot crosses to the engine for a single turn.

Explicit non-goals: the assistant is strictly informational—it executes **no** banking transactions, holds **no** direct database connection from the AI engine, and never indexes personal customer records.

> DESIGN PRINCIPLE **Evidence-driven engineering discipline**
> Every architectural and retrieval change followed a strict **step-and-plan protocol**: written hypothesis, offline structural verification, paired benchmark evaluation on frozen question sets, and zero-regression CI gating before any default was promoted.

SECTION 2

2

Platform Architecture & Governance

A strict two-system architecture separated by a single authenticated trust boundary: Bank Assistant owns identity, persistence and client policy; RAGLab owns retrieval, grounding and verification.

2.1 End-to-End Topology & Division of Ownership

Early iterations of the web application contained an embedded Python agent writing directly to PostgreSQL alongside Spring Boot, creating dual-schema writers and inconsistent refusal behaviour. Consolidating around **RAGLab as the sole answering engine** and **Spring Boot as the sole database writer** established clean operational ownership (Figure 2.1, Table 2.1).

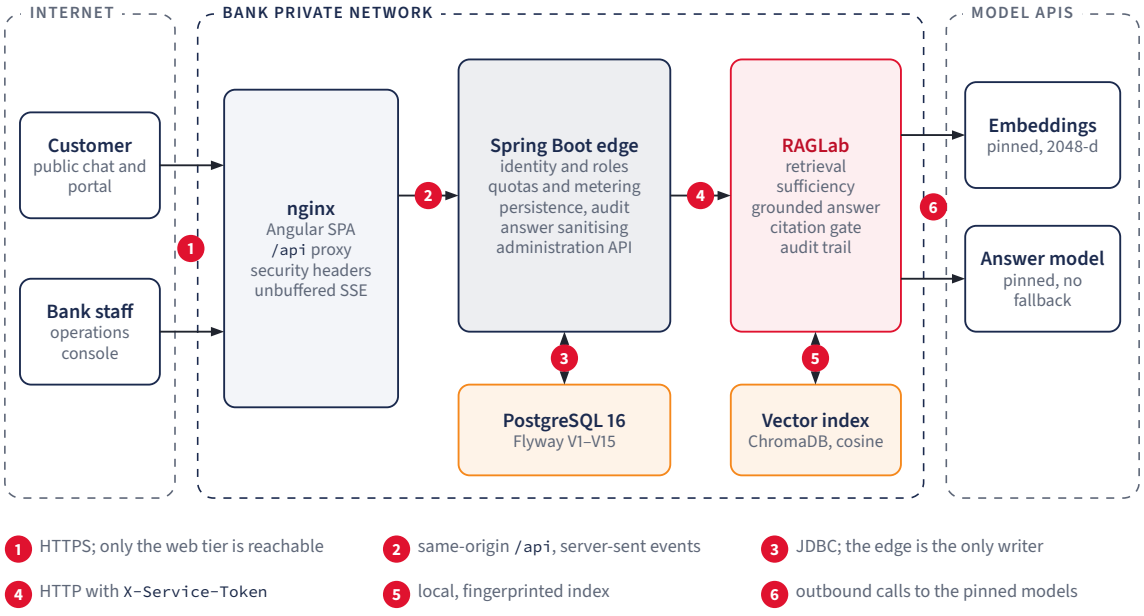


Figure 2.1 End-to-end platform topology across trust zones.

Table 2.1 Strict division of responsibilities across the edge–engine boundary.

Domain	Edge Service (`Bank Assistant`)	Answering Engine (`RAGLab`)
Identity & RBAC	Customer (`clients`) & staff (`users`) auth, JWT rotation, 9 permissions	Stateless; validates constant-time `X-Service-Token`
Persistence & state	Sole writer to PostgreSQL 16 (Flyway migrations); conversation history	Zero DB access; owns local ChromaDB & SHA-256 doc store
Customer context	Builds a one-request masked profile (ranges/counts, zero RIB/CIN)	Consumes transient masked block; never stores or indexes it
Provenance & refusals	Strips sources/citations for clients; substitutes bank refusal wording	Returns structured claims, quotes, source IDs & refusal reason
Quotas & resilience	Two-phase token/message metering under 64 striped session locks	Bounded model calls per turn; ingest concurrency seal

2.2 Lifecycle of a Customer Question & Core Technology Stack

Figure 2.2 traces a live question across all four runtime containers (`frontend` Nginx/Angular 21, `backend` Spring Boot 4 / Java 21 virtual threads, `postgres` PostgreSQL 16, and `raglab` FastAPI / Python 3.11).

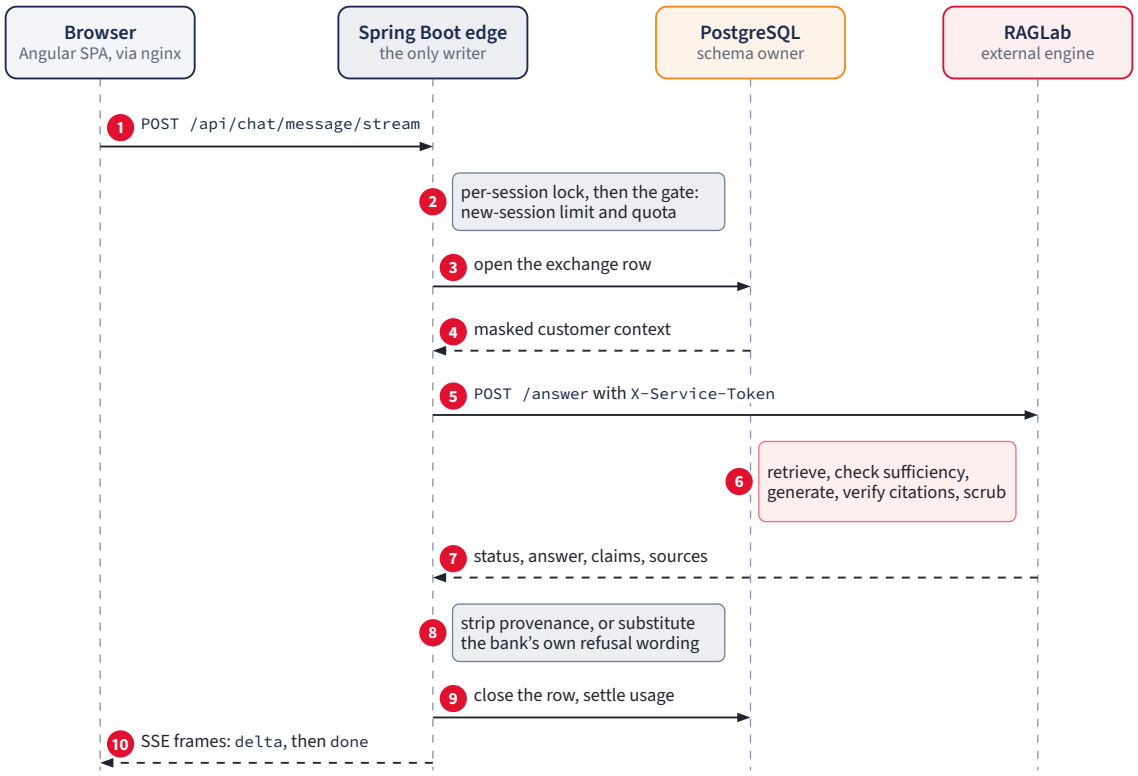


Figure 2.2 End-to-end sequence of a customer chat request across the four services.

Sober technology choices: To keep banking audits transparent and on-premise deployment lightweight, `RAGLab` avoids opaque orchestration frameworks (no LangChain/LlamaIndex) and vendor SDKs, using explicit Python modules, `urllib` HTTP clients, `tiktoken`, `pypdf`/OOXML loaders and local `ChromaDB`. On the web side, `PostgreSQL 16` schema evolution is strictly governed by `Flyway` migrations validated at boot (`ddl-auto: validate`), and `Nginx` serves `Angular 21` on a single origin (`/api` proxy with unbuffered SSE streaming), eliminating cross-origin exposure.

Table 2.2 Four-container production topology orchestrated by Docker Compose.

Container	Runtime Stack	Network	Operational Role
`frontend`	Angular 21 + Nginx 1.27	`8080:80`	Serves SPA, security headers, `/api` & SSE reverse proxy.
`backend`	Spring Boot 4 · Java 21	Internal `:8080`	Auth/JWT, RBAC, session quotas, masking, sole DB writer.
`postgres`	PostgreSQL 16 + Flyway	Internal `:5432`	15 SQL migrations (`V1–V15`), check constraints, audit rows.
`raglab`	FastAPI · Python 3.11	Internal `:8000`	ChromaDB index, retrieval, grounding gate & JSONL audit.

SECTION 3

3

RAGLab: The Four-Layer Answering Engine

Organised into four deterministic layers around a pinned embedding and generation core: corpus repair and structural chunking, declarative legal knowledge, request understanding, and citation-gated delivery.

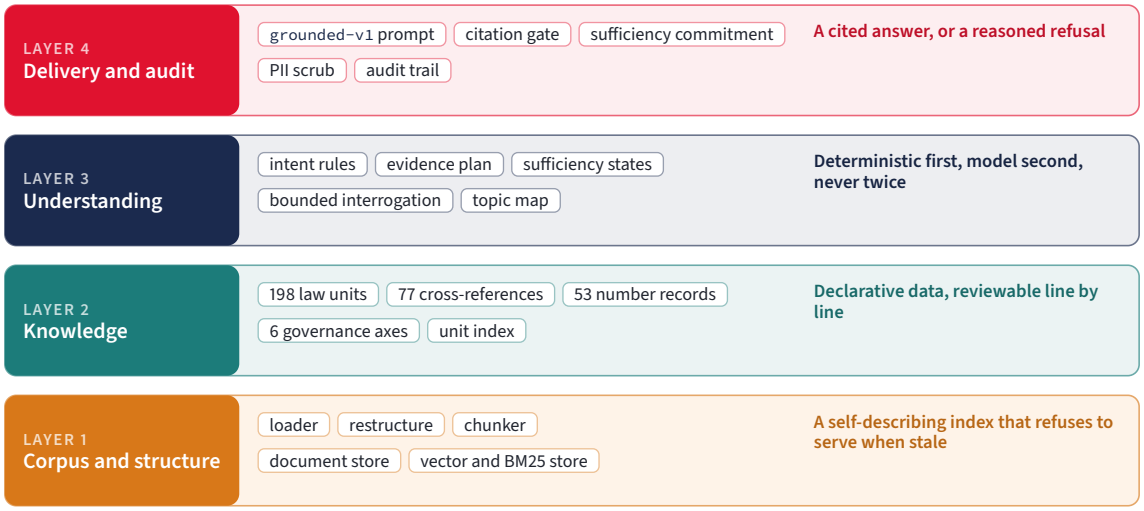


Figure 3.1 The four governed layers of RAGLab and the invariant guaranteed by each layer.

3.1 Layer 1 — Linguistic Corpus Repair, Structural Chunking & Self-Describing Index

Raw PDF extraction of Arabic legal texts suffered from reversed visual line ordering, broken article headers in Law n° 2016-48, and 28 corrupted digits in Circular n° 2019-08. Rather than letting an algorithm guess missing text, **294 deterministic linguistic repairs** were logged, verified by automated conservation checks, and approved before indexing (215 on the Law restoring all 198 articles, 35 on the Circular, 23 on the Internal Guide, and 21 on the Primer).

To preserve legal boundaries, the default `restructure` chunker (220-token budget, 40-token overlap) performs three operations: (1) strips repeated gazette headers and reconstructs RTL visual-order lines, (2) injects hierarchical `h1 > h2 > h3` breadcrumbs above sub-clauses and tables so no chunk loses its legal scope, and (3) splits recursively along structural headings. Shared Arabic normalisation (Unicode NFKC, presentation-form folding, *alif* unification, *tatweel* and diacritic removal) aligns indexing and querying.

Index safety guards: Every ChromaDB collection encodes a cryptographic fingerprint of its chunking mode, token budget, overlap and tokenizer; mismatched queries fail closed before any API call. Document uploads are deduplicated by `SHA-256` content hash, and an **ingest seal** blocks concurrent reads with an explicit `409 Conflict` while re-indexing runs.

3.2 Layer 2 — Declarative Knowledge Layer on Law n° 2016-48

Built deterministically on the Banking Law, Layer 2 structures regulatory text into versioned, automatically audited data artifacts:

- **198 stable knowledge units** (`loi-2016-48:art001` to `art198`) with verified 100 % article coverage, hierarchical paths and functional types;
- **77 cross-reference relations** (plus Circular-to-Article-11 anchoring) verified against literal text;
- **53 structured regulatory number records** (14 rates, 28 deadlines, 5 ceilings, 6 sanctions) exposed via a dedicated read-only API;

- **Six governance metadata axes** (authority, validity, scope, audience, domain, source) plus an isolated 198-unit vector collection that achieves **100 % rank-1 accuracy** on legal queries.

3.3 Layers 3 & 4 — Sufficiency, Citation Gate, Refusal Taxonomy & Service Contract

Incoming queries pass through local privacy guards (blocking account/balance lookups with zero model calls), intent classification, and optional evidence sufficiency checks. Generation uses a strict JSON schema (`claims[]`, cited `source_id` and evidence quotes, plus deterministic numeric and source-grounding verification) within a 3,000-token context window where whole chunks are never truncated mid-sentence.

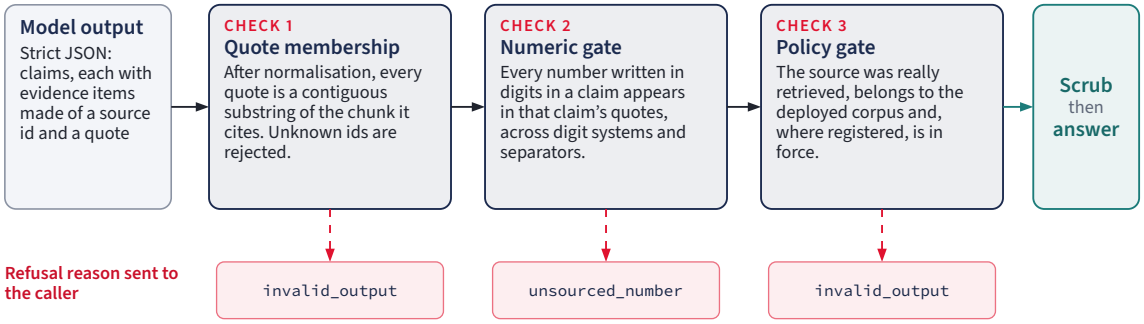


Figure 3.2 Post-generation verification gate: source validity, grounding check and deterministic number check.

Every draft must pass the deterministic gate (Figure 3.2): unknown sources, ungrounded claims, or **any number absent from the cited source passage** (`unsourced_number`) trigger immediate refusal. Outbound text is scrubbed for PII patterns (RIB, IBAN, CIN, phone, email), logged to a bounded 500-entry JSONL audit journal, and served through a frozen **27-route FastAPI contract** (including live runtime `.env` API-key management without container restarts).

Table 3.1 The six deterministic refusal codes exposed to staff in the audit console.

Refusal Code	Enforced By	Operational Meaning for Bank Supervisors
`private_or_live_request`	Local guard (0 calls)	Customer asked for live account, card, or transaction data.
`no_context`	Retrieval layer	No indexed chunk matched the request.
`evidence_insufficient`	Pre-generation gate	Retrieved excerpts lack the required evidentiary structure.
`insufficient_evidence`	Answer model	Model abstained cleanly because excerpts do not answer the query.
`unsourced_number`	Citation gate	Draft answer contained a digit/rate absent from its cited source.
`invalid_output`	Citation gate	Malformed JSON, unknown `source_id`, or policy violation.

SECTION 4

4

Bank Assistant: Edge, Portal & Console

Delivering governed answers safely to two distinct populations: a trilingual RTL-capable experience for customers and a full-provenance operational console for bank staff.

4.1 Identity Separation, Two-Phase Quota Metering & Fault-Tolerant Logging

Customers (`clients`, fixed marker `UTILISATEUR`) and staff (`users`, dynamic RBAC roles granted solely by the Super Admin / General Manager) are separated at both application and PostgreSQL constraint levels (Flyway migration `V15`). The customer portal is strictly read-only—displaying accounts, cards, movements, financings, a governed Islamic financing simulator, and advisor callback/appointment requests in Arabic, French and English.

To prevent quota races on Java 21 virtual threads without pinning carrier threads, the chat pipeline executes under **64 striped `ReentrantLock` instances** (Figure 4.1):

1. **Phase 1 (Admission Gate):** verifies IP new-session rate limits (5/hour anonymous) and session budgets (**3,000 tokens / 20 messages** for anonymous visitors; **10,000 tokens / 50 messages** for signed-in customers) *before* invoking `RAGLab`.
2. **Pre-call persistence:** opens the database exchange row prior to calling the engine so timeouts or incidents remain visible to supervisors as `FAILED` rows rather than vanishing.
3. **Phase 2 (Settlement & Sanitisation):** strips all internal document names and citation tags for customers, substitutes Al Baraka's official trilingual refusal wording when refused, records latency and token usage, and streams verified SSE frames (`delta` + `done`).

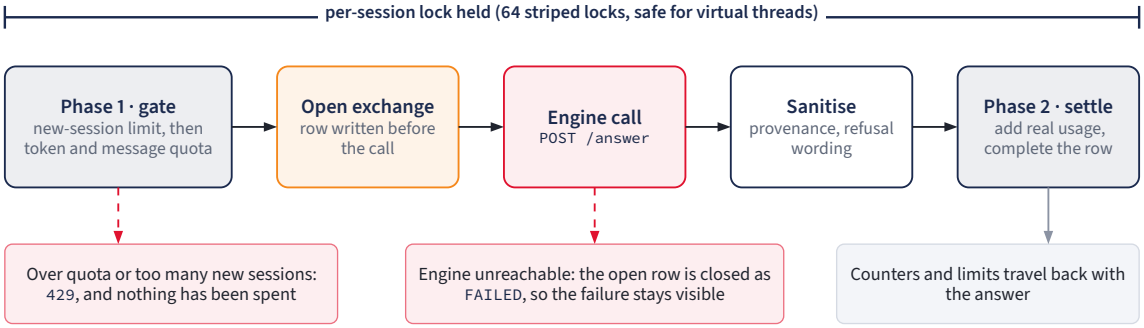


Figure 4.1 Two-phase session quota metering and pre-call audit row lifecycle on the Spring Boot edge.

4.2 Ephemeral Customer Context Masking & Staff Back-Office Console

When an authenticated customer asks a question, raw account numbers, CINs, emails and exact balances **never** leave the edge. Instead, `Bank Assistant` builds a transient, coarse-grained profile (customer segment, account categories, financing types, broad balance bands) valid for that single HTTP call (Figure 4.2).

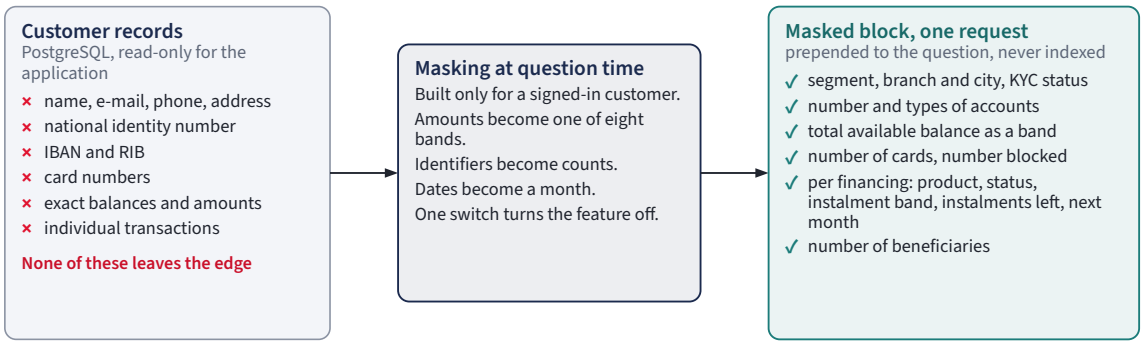


Figure 4.2 Privacy boundary: sensitive banking records stay in PostgreSQL while only a masked summary crosses to RAGLab.

On the staff side, a 9-permission RBAC model (`USERS`, `ROLES`, `CONVERSATIONS`, `DOCUMENTS`, `ANALYTICS`, `NEWS`, plus `RAGLAB_VIEW / RUN / MANAGE`) governs the French administration console. Authorised staff supervise conversations with full claim-by-claim source excerpts, monitor engine health and corpus synchronisation, run evaluations, and manage provider API keys (`.env` hot-reload) directly from `/admin/raglab/settings`.

Table 4.1 Separation of capabilities across customer-facing and staff-only web surfaces.

Surface	Target Audience	Key Governed Capabilities
Public Landing & Chat	Prospective visitors	Trilingual product discovery (AR/FR/EN, RTL) & quota-guarded Q&A.
Customer Portal	Signed-in `clients`	Read-only accounts, cards, movements, financings, simulator & callbacks.
Supervision Console	Support & Audit staff	Full conversation replay with claim quotes, source IDs & refusal codes.
RAGLab Admin Hub	`RAGLAB_*` roles	Live `/health`, corpus sync, benchmark runner & `.env` API-key hot-reload.

SECTION 5

5

Empirical Evaluation & Quality Gates

Every active default in the platform was earned through paired offline and live benchmarks on frozen trilingual question sets under an explicit non-regression rule.

5.1 Benchmark Protocol & Structure-Aware Chunking Gain (+20 pts hit@1)

Evaluation rests on two version-locked benchmarks: a **50-question main set** (45 evaluable across 10 verbatim, 20 paraphrases, 15 French/English cross-lingual queries + 5 out-of-scope refusal checks; spanning all 4 documents in AR/FR/EN) and a **10-question Arabic target set** testing colloquial, implicit and compound phrasing. No default is changed unless it improves accuracy without regressing on verbatim or out-of-scope cases.

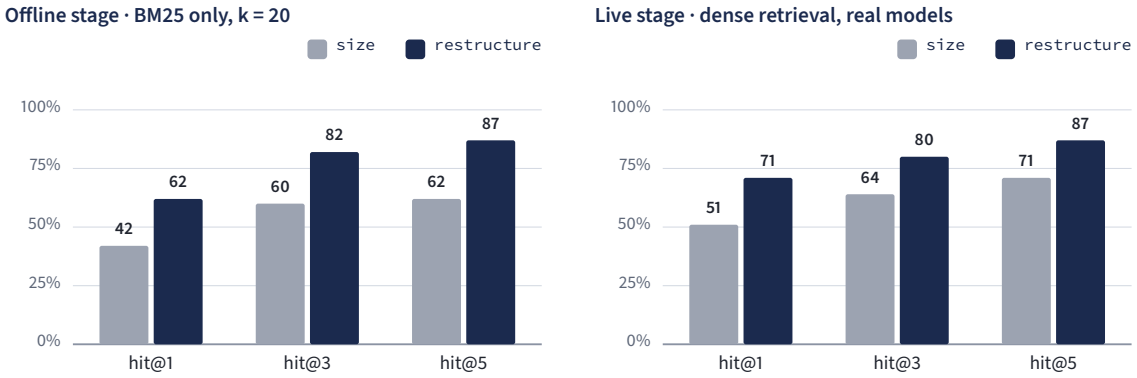


Figure 5.1 Paired retrieval accuracy on the 45 evaluable questions: fixed-window (`size`) vs. structure-aware (`restructure`) chunking.

Table 5.1 Live Nemotron retrieval accuracy: fixed window → structure-aware chunking.

Question Category	Count	Live hit@1	Live hit@3	Live hit@5
Verbatim Arabic	10	50 % → 50 %	50 % → 60 %	70 % → 80 %
Paraphrased banking queries	20	45 % → **80 %**	65 % → **90 %**	65 % → **95 %**
Cross-lingual (French & English)	15	60 % → 73 %	73 % → 80 %	80 % → 80 %
All Evaluable Questions	**45**	**51 % → 71 %**	**64 % → 80 %**	**71 % → 87 %**

Structure-aware chunking delivers a **+20 point jump in live hit@1** (51 % to 71 %) and a **+35 point surge on paraphrases** (45 % to 80 %), confirming that intact legal clauses with heading breadcrumbs align far better with natural customer wording than arbitrary token windows. Adding the deterministic reranker further lifted offline hit@3 from 82 % to 89 % with zero regressions.

5.2 Dense vs. Hybrid Retrieval, Legal Knowledge Index & Answer-Model Independence

Contrary to generic industry folklore, **hybrid BM25+vector fusion underperformed pure dense retrieval** on this asymmetric corpus (Figure 5.2): RRF dropped hit@5 from 87 % to 82 % (and French hit@1 from 50 % to 33 %), while score blending dropped overall hit@1 from 71 % to 62 %, because French queries share almost no lexical tokens with Arabic statutes—injecting lexical noise into rank fusion. Dense retrieval therefore remains the default.

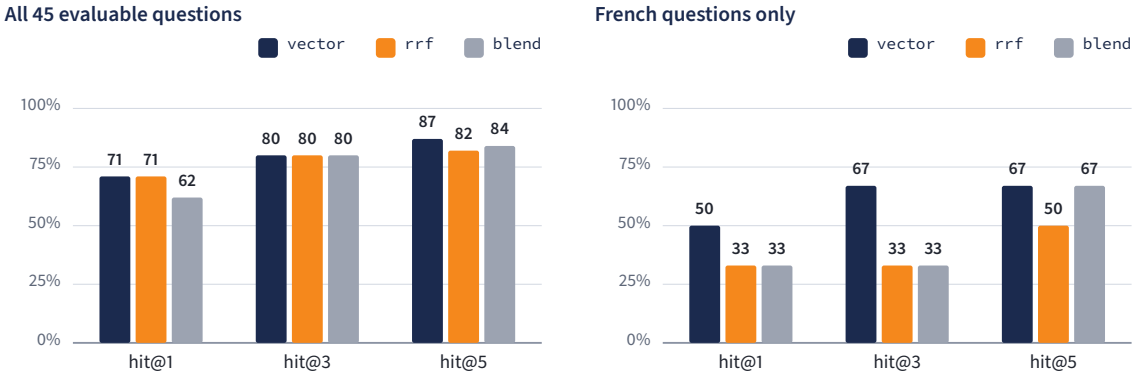


Figure 5.2 Live retrieval accuracy across modes: dense vector search outperforms RRF and score blending, especially on French queries.

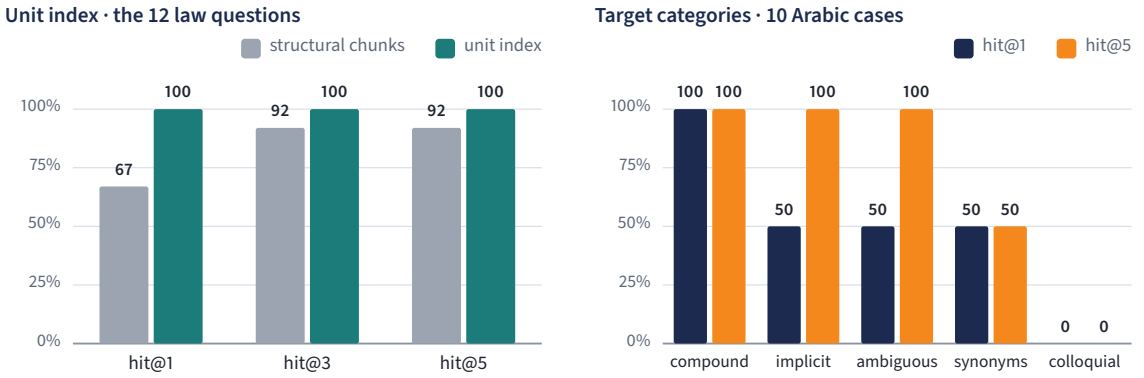


Figure 5.3 Left: the 198-unit Law n° 2016-48 index places 100 % of legal questions at rank 1. Right: performance on the 10-question Arabic target set.

Three further campaigns validate end-to-end reliability:

1. **Knowledge-unit precision (Figure 5.3):** indexing the 198 declarative units of Law n° 2016-48 raises legal retrieval from 67 % to **100 % at rank 1** (12/12 questions).
2. **Model independence:** feeding identical retrieved chunks to two distinct LLMs (`qwen/qwen3.8-max:free` vs. `moonshotai/kimi-k3`) yielded a **1.000 overlap of cited sources**, proving that grounding is governed by the retrieval and verification contract rather than model idiosyncrasies.
3. **Adversarial gate & CI suites: 6/6** prompt-injection, fabricated-number and out-of-policy citation attacks were deterministically rejected. Continuous integration enforces **472+ unit tests and 133 offline checks** on `RAGLab`, **214 backend/frontend tests** plus real PostgreSQL Flyway verification on `Bank Assistant`, and a **20-point 4-container Docker Compose smoke suite**.

SECTION 6

6

Security, Operations & Pilot Roadmap

Defence-in-depth controls across six trust boundaries, transparent personal-data compliance assessment under Tunisian law, one-command Docker delivery, and the roadmap to a supervised bank pilot.

6.1 Trust Boundaries, OWASP LLM Defences & Tunisian Personal-Data Assessment

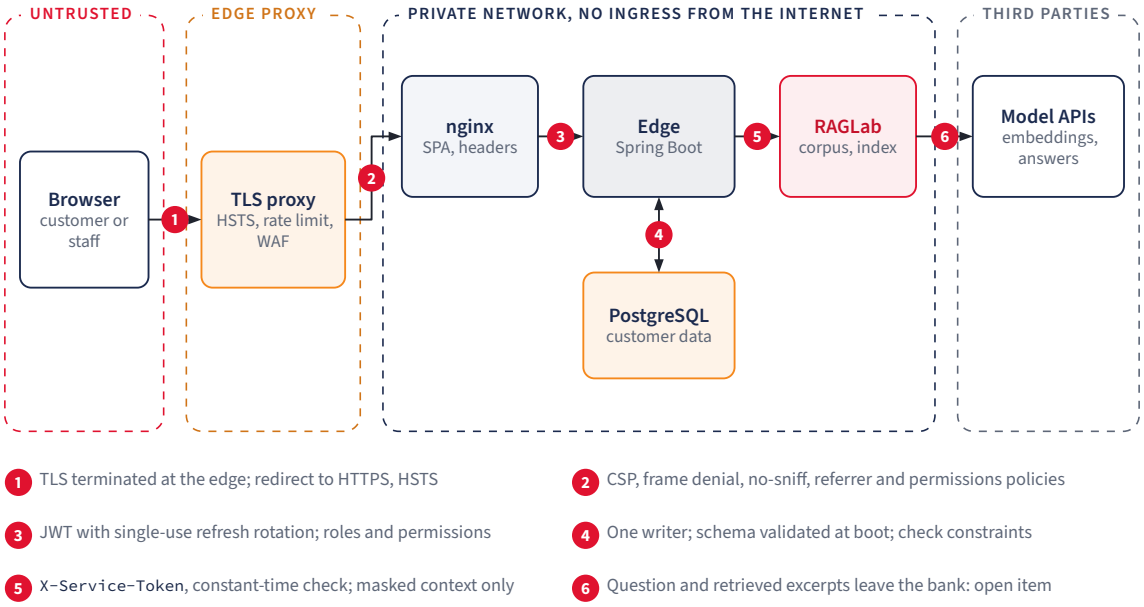


Figure 6.1 Platform trust boundaries and security controls enforced at each crossing.

Security is enforced in layers across Figure 6.1: strict CSP, `X-Frame-Options: DENY` and uncached `index.html` at Nginx; `BCrypt` hashing, typed JWTs with single-use rotated refresh families, per-request DB permission resolution, and non-PII structured ECS logs at Spring Boot; constant-time `X-Service-Token` validation, zero tool/agency execution, deterministic numeric/quote verification, and regex PII output scrubbing in `RAGLab`.

Against the **OWASP Top 10 for LLM Applications (2025)**, prompt injection is neutralised by treating excerpts as untrusted data and requiring verifiable source grounding; excessive agency is eliminated by design (zero tools or write APIs); sensitive disclosure is mitigated by index isolation and masked context; and unbounded consumption is capped by two-phase session quotas and per-route timeouts.

In accordance with **Tunisian Organic Law n° 2004-63 (INPDP)** and banking secrecy, a closing audit catalogued remaining pre-production gaps transparently (Table 6.1) so bank compliance and IT teams have an exact checklist prior to external customer exposure.

Table 6.1 Consolidated pre-pilot security and INPDP personal-data checklist.

Priority	Open Item (Audit ID)	Required Action Before Customer-Facing Pilot
High	Input-side PII & external APIs (`H-1`, `H-4`)	Scrub/reject CIN, RIB and phone numbers *before* outbound LLM/embedding calls, or deploy models on internal bank GPUs under a formal DPA.
High	Chat retention & IP masking (`PII-1..3`)	Enable scheduled retention/purge jobs, scrub free-text PII at rest in `messages`, and mask client IP addresses in monitoring list views.
High	Mandatory prod secrets (`H-3`, `P-2`)	Enforce non-default `JWT_SECRET`, mandatory `RAGLAB_SERVICE_TOKEN`, and `DEV_SEED=false` via startup assertions in the `prod` profile.
Medium	Embed cache & RBAC feed (`H-2`, `PII-5`)	Disable 120-char plaintext previews in `RAGLab` embedding caches and restrict raw question text visibility for read-only operators.

6.2 Containerised Handover Package & Operational Lessons

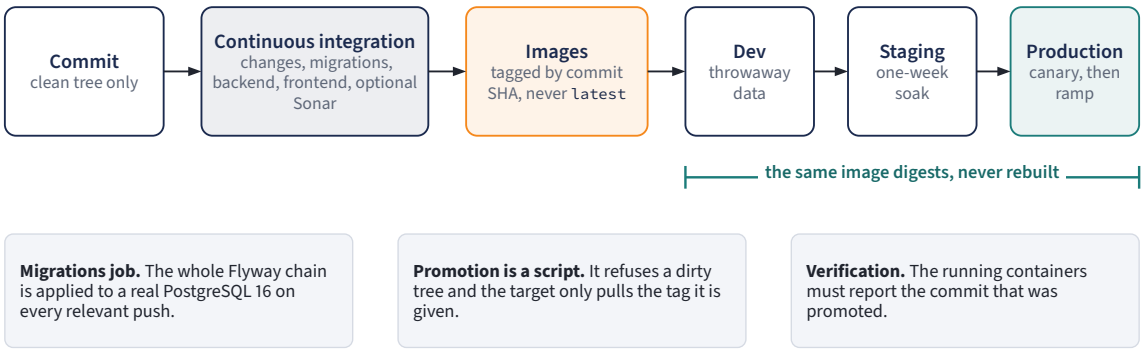


Figure 6.2 Container build, registry publication and environment promotion workflow.

The unified monorepo ships with automated GitHub Actions CI/CD publishing multi-service images (`elayarifarah/albaraka-raglab`, `albaraka-backend`, `albaraka-frontend`) to Docker Hub and a standalone handover archive (`albaraka-demo.zip`) requiring **no source code or local compilers**:

```
# Launch all 4 containers (PostgreSQL 16 + RAGLab + Spring Boot Edge + Angular/Nginx)
docker compose pull && docker compose up -d
# Portal & console: http://localhost:8080  |  Actuator & Prometheus metrics enabled
```

Real-world integration testing uncovered and resolved critical operational defects that unit tests alone missed—including a SHA-256 content-hash guard preventing duplicate corpus uploads, historical conversation provenance sanitisation on replay, Flyway Boot 4 starter packaging assertions, and permission-aware landing routes for custom staff roles.

6.3 Recommended Closing Roadmap for Al Baraka Bank Tunisia

1. **Phase A — Internal Supervised Staff Pilot (Immediate):** Deploy the containerised stack on an internal Al Baraka server for branch advisors and compliance officers (`DEV_SEED=false`, internal TLS reverse proxy). Staff benefit immediately from instant, article-cited lookups across Law n° 2016-48, Circular n° 2019-08 and the Internal Operations Guide.
2. **Phase B — Pre-Customer Compliance Gate (Short Term):** Implement input-side regex PII scrubbing (`H-1`), activate automated conversation retention purging, and finalise INPDP / sub-processor governance—or point `RAGLab`'s provider adapter to bank-hosted local inference endpoints.
3. **Phase C — Domain Expansion (Medium Term):** Inject governed bank rates (`business_facts`) directly into the answering context, expand the frozen evaluation benchmark with branch-authored French/Arabic inquiries, and add role-scoped document collections for multi-department roll-out.